

Universidad Nacional de Moquegua

Facultad de Ingenierías

Escuela Profesional de Ingeniería de Sistemas e Informática



IS-722 Calidad de Software

Informe Técnico

ISO/IEC 27001:2022 — Controles 8.25, 8.28 y 8.29

Ciclo: 2026 - I

Estudiantes:

Harol Gerardo Mendoza Segura

Juan Edwin Calizaya Llanos

Joel Sebastián Luna Luna

Guido Maron Acero

Fernando Manuel Condori Apaza

Eduardo Paolo Condori Chambi

Docente:

Mgr. Juan Carlos Valero Gómez

Índice

1	Introducción	4
1.1	La necesidad de seguridad en el desarrollo de software	4
1.2	ISO/IEC 27001:2022 — Visión general	4
1.3	Controles tecnológicos — 8.25, 8.28 y 8.29	4
1.4	Objetivo del informe	5
1.5	Alcance — Caso de estudio: Workcodile-dev	5
1.6	Estructura del informe	5
2	Secure Software Development Life Cycle (SSDLC)	6
2.1	Qué intenta resolver	6
2.2	Qué es realmente un SSDLC	6
2.3	Fase 1: Requisitos de Seguridad	7
2.3.1	Activos de información	7
2.3.2	Clasificación de la información	7
2.3.3	Requisitos de seguridad	8
2.3.4	Requisitos regulatorios	8
2.3.5	Requisitos de autenticación y acceso	8
2.4	Fase 2: Diseño Seguro	8
2.4.1	Threat Modeling con STRIDE	8
2.4.2	Diagrama de Flujo de Datos (DFD)	8
2.4.3	Límites de confianza	9
2.4.4	Controles de diseño	9
2.5	Fase 3: Desarrollo Seguro	9
2.6	Fase 4: Pruebas de Seguridad	10
2.7	Fase 5: Despliegue Seguro	10
2.7.1	Hardening de servidores y contenedores	10
2.7.2	Gestión de secretos	10
2.7.3	Seguridad CI/CD	11
2.8	Fase 6: Mantenimiento	11
2.9	Ciclo PDCA del SGSI	11
2.10	Lo que busca un auditor en 8.25	11
3	Secure Coding	12
3.1	Filosofía	12
3.2	Principios Fundamentales de Codificación Segura	12
3.2.1	1. Nunca confiar en la entrada	12
3.2.2	2. Validar siempre	13
3.2.3	3. Sanitizar cuando corresponda	13
3.2.4	4. Principio de mínimo privilegio	14
3.2.5	5. Gestión de secretos	14
3.2.6	6. Manejo seguro de errores	14
3.2.7	7. Dependencias seguras	15
3.3	OWASP Top 10 y el control 8.28	15
3.4	Qué busca un auditor en 8.28	15
4	Security Testing	16
4.1	Tipos de Pruebas de Seguridad	16

4.1.1	SAST — Static Application Security Testing	16
4.1.2	DAST — Dynamic Application Security Testing	16
4.1.3	Diferencia clave entre SAST y DAST	17
4.1.4	Code Review — Revisión de Código	17
4.1.5	Vulnerability Assessment	18
4.1.6	Penetration Testing	18
4.2	Acceptance Testing de Seguridad	18
4.3	Pipeline CI/CD Seguro	19
4.4	Qué busca un auditor en 8.29	19
5	Métricas de Calidad	19
5.1	Métrica 1: Densidad de Vulnerabilidades	19
5.2	Métrica 2: Cobertura de Análisis SAST	20
5.3	Métrica 3: Tiempo Medio de Remediación	20
5.4	Métrica 4: Tasa de Aceptación de Seguridad	21
5.5	Métrica 5: Nivel de Madurez del SSDLC	21
5.6	Resumen de Métricas	22
6	Aplicación Práctica y Conclusiones	22
6.1	Contexto del proyecto	22
6.2	Aplicación de SSDLC (8.25)	22
6.2.1	Activos de información	22
6.2.2	Clasificación de la información	22
6.2.3	Requisitos de seguridad (CIA + T)	23
6.2.4	Threat Modeling con STRIDE	23
6.2.5	Diagrama de Flujo de Datos (DFD)	23
6.3	Aplicación de Secure Coding (8.28)	24
6.3.1	Principios aplicados	24
6.3.2	OWASP Top 10 — Mapeo	25
6.4	Aplicación de Security Testing (8.29)	25
6.4.1	Herramientas SAST	25
6.4.2	Herramientas DAST	25
6.4.3	Vulnerability Assessment	26
6.4.4	Pipeline CI/CD de Workcodile-dev	26
6.5	Métricas aplicadas a Workcodile-dev	26
6.6	Conclusiones	26
6.7	Recomendaciones	27
6.8	Lo que busca un auditor en 8.25, 8.28 y 8.29 — Resumen	27
	Referencias	27

1. Introducción

1.1. La necesidad de seguridad en el desarrollo de software

La industria del software enfrenta un panorama de amenazas creciente. Cada año, las vulnerabilidades explotadas en producción generan pérdidas económicas multimillonarias, daño reputacional y exposición de datos sensibles de millones de usuarios. Según el reporte *OWASP Top 10* de 2021, las vulnerabilidades más críticas siguen siendo aquellas que nacen en la fase de implementación del software: inyecciones de código, fallos de control de acceso y configuraciones inseguras [OWASP Foundation \(2021\)](#).

Frente a esta realidad, los organismos internacionales de estandarización han desarrollado normas que proporcionan un marco sistemático para gestionar la seguridad de la información a lo largo del ciclo de vida del desarrollo de software. La norma **ISO/IEC 27001:2022** es la más adoptada internacionalmente.

1.2. ISO/IEC 27001:2022 — Visión general

La norma **ISO/IEC 27001:2022** establece los requisitos para establecer, implementar, mantener y mejorar continuamente un **Sistema de Gestión de Seguridad de la Información (SGSI)**. Es la norma internacional de referencia para la gestión de seguridad de la información y es aplicable a cualquier organización, independientemente de su tamaño, sector o complejidad [Wikipedia \(2026\)](#).

La norma se estructura en dos partes principales:

1. **Cláusulas 4 a 10:** Requisitos del sistema de gestión. Definen el marco de trabajo que la organización debe establecer, incluyendo contexto organizacional, liderazgo, planificación, apoyo, operación, evaluación del desempeño y mejora continua.
2. **Anexo A:** Controles de referencia. Lista de 93 controles de seguridad organizados en 4 temas:
 - **A.5:** Controles organizacionales (37 controles).
 - **A.6:** Controles de personas (8 controles).
 - **A.7:** Controles físicos (14 controles).
 - **A.8:** Controles tecnológicos (34 controles).

ISO/IEC 27001 no es un conjunto de “recetas” o herramientas específicas. Es un estándar basado en procesos que define qué se debe lograr, dejando a cada organización la libertad de decidir cómo implementar los controles según su contexto y necesidades específicas.

1.3. Controles tecnológicos — 8.25, 8.28 y 8.29

Los controles más directamente relacionados con el desarrollo de software en ISO/IEC 27001:2022 pertenecen a la categoría de controles tecnológicos (A.8).

Establecen requisitos específicos para garantizar que la seguridad sea parte integral del proceso de desarrollo, no una verificación posterior. Los tres controles principales son:

Control	Descripción
8.25	Secure Development Life Cycle — Reglas para el desarrollo seguro
8.28	Secure Coding — Principios de codificación segura
8.29	Security Testing in Development and Acceptance — Pruebas de seguridad

Cuadro 1: Controles tecnológicos de ISO/IEC 27001:2022

El control 8.25 exige que la organización establezca reglas para el desarrollo seguro de software y sistemas, aplicándolas durante todo el ciclo de vida. El control 8.28 se enfoca en los principios de codificación segura para reducir la introducción de vulnerabilidades durante la implementación. El control 8.29 requiere que existan procesos de prueba de seguridad definidos e implementados en el ciclo de vida del desarrollo [Wikipedia \(2026\)](#).

1.4. Objetivo del informe

Este informe explica los controles 8.25, 8.28 y 8.29 de ISO/IEC 27001:2022 y cómo aplicarlos al desarrollo de software. Se usa el proyecto **Workcodile-dev** como ejemplo real.

1.5. Alcance — Caso de estudio: Workcodile-dev

Workcodile-dev es una plataforma web full-stack para estudiantes de la Universidad Nacional de Moquegua. Su arquitectura permite analizar la aplicación de los controles de seguridad:

Capa	Tecnología
Frontend	React 18, TypeScript, Vite, Tailwind CSS
Backend	Node.js, Express.js, JWT, bcrypt
Base de datos	MongoDB (NoSQL)
Almacenamiento	MinIO (compatible S3)
Infraestructura	Docker, Docker Compose

Cuadro 2: Stack tecnológico de Workcodile-dev

1.6. Estructura del informe

1. **Introducción** — Contexto, visión general de ISO/IEC 27001:2022 y los controles 8.25, 8.28 y 8.29.

2. **Secure Software Development Life Cycle (SSDLC)** — Explicación del ciclo de vida seguro y el control 8.25, con relación a 8.28 y 8.29.
3. **Secure Coding** — Principios de codificación segura según el control 8.28, con ejemplos prácticos.
4. **Security Testing** — Metodologías de prueba de seguridad según el control 8.29.
5. **Métricas de Calidad** — Métricas cuantitativas para medir el cumplimiento de la norma.
6. **Aplicación Práctica y Conclusiones** — Caso de estudio Workcodile-dev, hallazgos y recomendaciones.

2. Secure Software Development Life Cycle (SSDLC)

El control 8.25 de ISO/IEC 27001:2022 establece que la organización debe establecer reglas para el desarrollo seguro de software y sistemas, aplicándolas durante todo el ciclo de vida del desarrollo. Este control se materializa a través del Secure Software Development Life Cycle (SSDLC), que integra actividades de seguridad en cada fase del ciclo de vida del desarrollo.

2.1. Qué intenta resolver

En el modelo tradicional de desarrollo de software, la seguridad solía revisarse al final del proyecto:

Análisis → Diseño → Desarrollo → Testing funcional → Producción



“Ahora revisemos seguridad”

Este enfoque provoca que los problemas de seguridad se detecten tarde, cuando ya existen más componentes afectados, más usuarios impactados y un costo de corrección mucho mayor. Las consecuencias típicas incluyen:

- Vulnerabilidades que aparecen en producción.
- Información sensible expuesta.
- Controles de acceso débiles o inconsistentes.
- Errores de implementación costosos de corregir.

Por eso, ISO 27001 exige que la seguridad esté presente desde el inicio del ciclo de vida del software, no como una verificación posterior.

2.2. Qué es realmente un SSDLC

Un SSDLC es un SDLC tradicional con actividades de seguridad integradas en cada fase. La idea principal es que la seguridad no se trate como una revisión

final, sino como un criterio transversal que acompaña todo el proceso de construcción del software [National Institute of Standards and Technology \(2022\)](#).

Fase	Pregunta clave	Control ISO/IEC 27001:2022
Requisitos	¿Qué debemos proteger?	8.25
Diseño	¿Cómo podrían atacarnos?	8.25
Desarrollo	¿Cómo codificamos de forma segura?	8.28
Pruebas	¿Cómo sabemos que es seguro?	8.29
Despliegue	¿Cómo nos protegemos en producción?	8.25
Mantenimiento	¿Cómo nos mantenemos seguros?	8.25

Cuadro 3: Relación de fases SSDLC con controles ISO/IEC 27001:2022

2.3. Fase 1: Requisitos de Seguridad

En esta fase se responde la pregunta: *¿Qué debemos proteger?*

2.3.1. Activos de información

Se deben identificar todos los activos de información del sistema que requieren protección. Los activos incluyen:

- Bases de datos y repositorios de información.
- Archivos y documentos del sistema.
- Servicios y APIs expuestas.
- Interfaces de programación.
- Credenciales y claves criptográficas.
- Registros de auditoría.
- Variables de entorno con secretos.
- Código fuente y configuraciones.

2.3.2. Clasificación de la información

Una vez identificados los activos, se clasifican según su nivel de sensibilidad:

Categoría	Acceso	Ejemplo típico
Pública	Sin restricción	Información institucional, catálogos
Interna	Empleados	Código fuente, configuraciones internas
Confidencial	Autorizados	Datos de usuarios, credenciales hasheadas
Restringida	Mínimo necesario	Claves maestras, secretos de producción

Cuadro 4: Clasificación de información según ISO/IEC 27001

2.3.3. Requisitos de seguridad

Los requisitos de seguridad se basan en el triángulo CIA + Trazabilidad:

- **Confidencialidad:** Solo usuarios autorizados acceden a la información.
- **Integridad:** Los datos no son modificados sin autorización.
- **Disponibilidad:** El servicio permanece operativo cuando sea necesario.
- **Trazabilidad:** Las acciones del sistema se registran y pueden auditarse.

2.3.4. Requisitos regulatorios

Dependiendo del proyecto, pueden existir exigencias asociadas a marcos normativos, políticas internas o leyes de protección de datos. En el caso de software educativo, pueden aplicar regulaciones de protección de datos personales.

2.3.5. Requisitos de autenticación y acceso

Se definen los mecanismos de:

- Autenticación: verificación de identidad del usuario.
- Autorización: control de acceso por roles o privilegios.
- Sesiones seguras: gestión de tokens y expiración.
- Control de acceso basado en contexto.

2.4. Fase 2: Diseño Seguro

En esta fase se responde la pregunta: *¿Cómo podrían atacarnos?*

2.4.1. Threat Modeling con STRIDE

La metodología STRIDE es una de las más utilizadas para identificar amenazas. Fue desarrollada por Microsoft y categoriza las amenazas en 6 tipos [OWASP Foundation \(2021\)](#):

2.4.2. Diagrama de Flujo de Datos (DFD)

Se identifican los componentes del sistema:

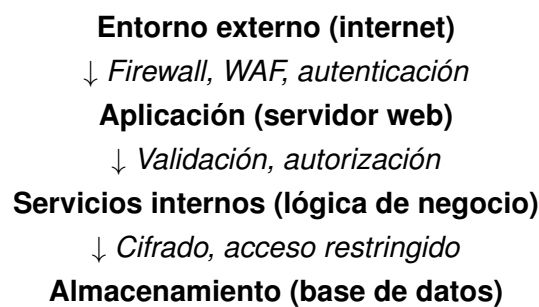
- **Procesos:** Componentes que transforman datos.
- **Flujos:** Movimiento de datos entre componentes.
- **Almacenes de datos:** Bases de datos, archivos.
- **Entidades externas:** Sistemas o usuarios externos.

Amenaza	Significado	Pregunta clave
Spoofing	Suplantación	¿Quién es realmente el usuario?
Tampering	Manipulación	¿Pueden alterarse los datos en tránsito?
Repudiation	Negación	¿Se puede demostrar quién hizo qué?
Info Disclosure	Exposición	¿Se puede leer información confidencial?
DoS	Indisponibilidad	¿Se puede interrumpir el servicio?
Elevation of Privilege	Escalada	¿Se pueden obtener permisos extra?

Cuadro 5: Metodología STRIDE para threat modeling

2.4.3. Límites de confianza

Los límites de confianza permiten distinguir qué componentes requieren controles adicionales:



2.4.4. Controles de diseño

Basado en el threat modeling, se definen controles como:

- Validación de autenticidad en cada punto de entrada.
- Autorización adecuada por componente.
- Registro de eventos relevantes (logging).
- Protección de sesiones y tokens.
- Segregación de componentes.

2.5. Fase 3: Desarrollo Seguro

Esta fase corresponde directamente al control 8.28 de ISO/IEC 27001:2022 (Secure Coding). El objetivo es garantizar que el software sea desarrollado utilizando principios de codificación segura para reducir la introducción de vulnerabilidades durante la implementación [Wikipedia \(2026\)](#).

La organización debe establecer procesos de codificación segura que incluyan:

- Principios de codificación segura aprobados.
- Requisitos de seguridad para desarrollo interno y externo.
- Procedimientos para prevenir vulnerabilidades conocidas.
- Uso de herramientas de desarrollo configuradas de forma segura.
- Entornos de desarrollo controlados y protegidos.

Los detalles de estos principios se desarrollan en la Sección 3 del presente informe.

2.6. Fase 4: Pruebas de Seguridad

Esta fase corresponde al control 8.29 de ISO/IEC 27001:2022 (Security Testing in Development and Acceptance). El control exige que las pruebas de seguridad formen parte del proceso normal de pruebas del sistema [Wikipedia \(2026\)](#).

Las verificaciones deben incluir:

- Funciones de autenticación y autorización.
- Controles de acceso.
- Gestión de sesiones.
- Registro y auditoría de eventos.
- Configuraciones de seguridad.

Las actividades de verificación incluyen SAST, DAST, revisiones de código y escaneo de vulnerabilidades. Esto se detalla en la Sección 4.

2.7. Fase 5: Despliegue Seguro

El despliegue seguro complementa el control 8.25. Los controles típicos incluyen:

2.7.1. Hardening de servidores y contenedores

- Uso de imágenes mínimas (ejemplo: `node:alpine` en lugar de `node:full`).
- Usuarios no root en entornos de producción.
- Configuración restrictiva de puertos.
- Eliminación de servicios innecesarios.

2.7.2. Gestión de secretos

- Variables de entorno en archivos `.env` (excluidos del control de versiones).
- Archivos `.env.example` como plantillas versionadas.
- Nunca secretos hardcodeados en código fuente.
- Uso de gestores de secretos para producción.

2.7.3. Seguridad CI/CD

- Control de acceso a repositorios.
- Protección de ramas principales mediante revisiones.
- Validación automática con pruebas antes de cada integración.
- Ejecución de análisis de seguridad dentro del pipeline.
- Segregación entre entornos de desarrollo, pruebas y producción.

2.8. Fase 6: Mantenimiento

El mantenimiento de seguridad es continuo e incluye:

- Actualización periódica de dependencias.
- Corrección de vulnerabilidades identificadas.
- Monitoreo continuo de riesgos y amenazas.
- Revisión de configuraciones de seguridad.
- Revisión de permisos y controles de acceso.
- Evaluación de nuevas vulnerabilidades publicadas.

2.9. Ciclo PDCA del SGSI

El SSDLC se enmarca dentro del ciclo de mejora continua del SGSI según ISO/IEC 27001. El ciclo Plan-Do-Check-Act (PDCA) asegura que los controles de seguridad se mejoren de forma continua [Wikipedia \(2026\)](#):

Fase	Actividad	Relación con SSDLC
Plan	Planificar	Definir requisitos de seguridad, threat modeling
Do	Implementar	Codificación segura, pipelines CI/CD
Check	Evaluar	SAST, DAST, auditorías de seguridad
Act	Mejorar	Corregir vulnerabilidades, actualizar procesos

Cuadro 6: Ciclo PDCA aplicado al SSDLC

2.10. Lo que busca un auditor en 8.25

Un auditor de ISO/IEC 27001 busca evidencias de:

- Política de Secure SDLC documentada.
- Capacitación del equipo en seguridad.
- Requisitos de seguridad definidos antes del desarrollo.

- Threat modeling realizado y documentado.
- Resultados de pruebas de seguridad.
- Control de cambios y pipeline seguro.
- Registros de mantenimiento de seguridad.

No busca una herramienta específica. Busca un proceso definido, aplicado y demostrable.

3. Secure Coding

El control 8.28 de ISO/IEC 27001:2022 establece que la organización debe establecer principios de codificación segura y aplicarlos al desarrollo de software para reducir la introducción de vulnerabilidades durante la implementación [Wikipedia \(2026\)](#).

3.1. Filosofía

Muchos ataques exitosos no son sofisticados. Son errores simples como SQL Injection, XSS, Path Traversal, Command Injection o CSRF. Todos nacen del código. Por eso ISO 27001 crea un control exclusivo para la programación segura.

El objetivo de 8.28 no es que cada programador sea un experto en seguridad, sino que existan reglas claras, documentadas y aplicables que reduzcan la superficie de ataque desde la fase de implementación.

3.2. Principios Fundamentales de Codificación Segura

La organización debe establecer principios de codificación segura aprobados. Los principios son:

3.2.1. 1. Nunca confiar en la entrada

La entrada del usuario siempre debe considerarse no confiable hasta que sea validada. Esto aplica a:

- Parámetros de URL y query strings.
- Cuerpos de peticiones HTTP (JSON, form-data).
- Headers y cookies.
- Archivos subidos por el usuario.
- Datos de bases de datos externas.

Ejemplo genérico de código inseguro:

Listing 1: Código inseguro: sin validación de entrada

```
// INSEGURO - No validación de entrada
app.get('/api/items/:id', (req, res) => {
```

```
const item = db.query('SELECT * FROM items WHERE id = ' + req.params.id);
res.json(item);
});
```

Solución genérica:

Listing 2: Código seguro: con validación de entrada

```
// SEGURO - Con validación y consulta parametrizada
app.get('/api/items/:id', (req, res) => {
  if (!isValidId(req.params.id)) {
    return res.status(400).json({ error: 'ID invalido' });
  }
  const item = db.query(
    'SELECT * FROM items WHERE id = ?', [req.params.id]
  );
  res.json(item);
});
```

3.2.2. 2. Validar siempre

Validar significa comprobar que el dato cumple lo esperado antes de procesarlo. Se deben validar: tipo, longitud, formato, rango y conjunto permitido de valores.

Listing 3: Ejemplo genérico de validación de entrada

```
const { body, validationResult } = require('express-validator');

app.post('/api/register', [
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 8 })
    .matches(/^(?=.*[A-Z])/),
  body('name').trim().escape()
    .isLength({ max: 100 }),
], (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      errors: errors.array()
    });
  }
  // Procesar registro...
});
```

3.2.3. 3. Sanitizar cuando corresponda

Sanitizar significa preparar los datos para su uso seguro. Un input como `<script>alert(1)</script>` puede producir Stored XSS si se almacena sin control.

En frameworks modernos como React, la salida se escapa automáticamente. Sin embargo, hay funciones que deben evitarse:

Listing 4: Uso inseguro de dangerouslySetInnerHTML

```
// INSEGURO - Nunca usar sin sanitizar
const UserBio = ({ bio }) => {
  return <div
    dangerouslySetInnerHTML={{ __html: bio }}
  />;
};

// SEGURO - React escapa automáticamente
const UserBio = ({ bio }) => {
```

```
return <div>{bio}</div>;  
};
```

3.2.4. 4. Principio de mínimo privilegio

El software debe usar únicamente los permisos que necesita para operar. Esto aplica a:

- **Bases de datos:** El usuario de la aplicación debe tener permisos solo sobre la base de datos de la aplicación, no sobre administración del servidor.
- **Almacenamiento:** Las credenciales de acceso a almacenamiento deben limitarse al bucket o directorio necesario.
- **Contenedores:** Los contenedores deben ejecutarse con usuarios no root y exponer solo los puertos estrictamente necesarios.
- **Archivos de configuración:** Los permisos de archivos `.env` deben ser restringidos al proceso de la aplicación.

3.2.5. 5. Gestión de secretos

Nunca almacenar secretos en código fuente ni en repositorios:

- Usar archivos de variables de entorno (ej: `.env`) excluidos del control de versiones.
- Crear archivos plantilla versionados (ej: `.env.example`) como referencia.
- Cargar variables de entorno con librerías dedicadas (ej: `dotenv`).
- Nunca: `const PW = "admin123"` en código.
- En producción: usar gestores de secretos como HashiCorp Vault, AWS Secrets Manager o Azure Key Vault.

3.2.6. 6. Manejo seguro de errores

Los mensajes de error no deben exponer información sensible del sistema:

Listing 5: Manejo seguro de errores

```
// INSEGURO - Expone detalles del error  
app.use((err, req, res, next) => {  
  res.status(500).json({  
    error: err.message,  
    stack: err.stack // NUNCA exponer  
  });  
});  
  
// SEGURO - Mensaje genérico, log interno  
app.use((err, req, res, next) => {  
  console.error(`${new Date()} ${err.message}`);  
  res.status(500).json({  
    error: 'Error interno del servidor'  
  });  
});
```

3.2.7. 7. Dependencias seguras

Gran parte de los ataques actuales no vienen del código propio, sino de dependencias externas. La organización debe establecer procesos para:

- Escanear dependencias periódicamente para vulnerabilidades conocidas.
- Actualizar paquetes con vulnerabilidades identificadas.
- Eliminar dependencias abandonadas o innecesarias.
- Usar herramientas automáticas de escaneo (ej: `npm audit`, `pip-audit`, `safety`).
- Definir una política de actualización de dependencias.

3.3. OWASP Top 10 y el control 8.28

El control 8.28 gira alrededor de prevenir las vulnerabilidades del OWASP Top 10 [OWASP Foundation \(2021\)](#). Estas son las categorías más relevantes:

OWASP	Nivel	Principio de prevención
Injection	Crítica	Validación y sanitización de entradas, consultas parametrizadas
Broken Access Control	Crítica	Control de acceso por roles, principio de mínimo privilegio
Cryptographic Failures	Alta	Cifrado en reposo y tránsito, hashing con <code>bcrypt/argon2</code>
Security Misconfiguration	Alta	Hardening de servidores, contenedores y servicios
Vulnerable Components	Alta	Escanear de dependencias, actualización periódica
Authentication Failures	Alta	Tokens con expiración, contraseñas hasheadas

Cuadro 7: Principios de prevención para las categorías OWASP Top 10

3.4. Qué busca un auditor en 8.28

Un auditor de ISO/IEC 27001 busca evidencias de:

- Documentación de principios de codificación segura aprobados.
- Guías de codificación segura disponibles para los desarrolladores.
- Plugins de seguridad en herramientas de desarrollo (linters, SAST).
- Revisiones de código con checklist de seguridad.
- Ausencia de secretos hardcodeados en el código fuente.
- Uso de librerías y dependencias actualizadas.

- Registros de escaneo de dependencias.

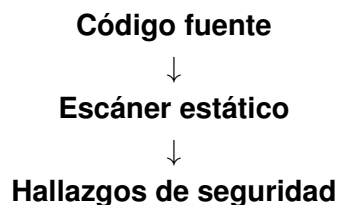
4. Security Testing

El control 8.29 de ISO/IEC 27001:2022 (Security Testing in Development and Acceptance) responde una pregunta clave: *¿Cómo sabemos que realmente es seguro?* Decir “seguimos buenas prácticas” no es evidencia. Hay que demostrarlo mediante pruebas [Wikipedia \(2026\)](#).

4.1. Tipos de Pruebas de Seguridad

4.1.1. SAST — Static Application Security Testing

El análisis estático de seguridad analiza el código fuente *sin ejecutarlo*. Detecta patrones conocidos de vulnerabilidades directamente en el código.



Qué busca:

- SQL Injection / NoSQL Injection.
- Hardcoded secrets (credenciales en código).
- Cross-Site Scripting (XSS).
- Uso inseguro de APIs.
- Errores de configuración en el código.

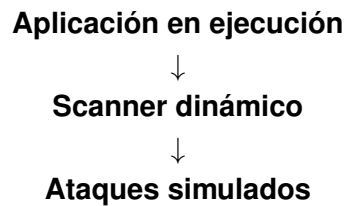
Herramientas comunes para SAST:

- **ESLint con plugins de seguridad:** Análisis estático de JavaScript/TypeScript con reglas específicas de seguridad.
- **Semgrep:** Análisis estático con reglas personalizables para múltiples lenguajes.
- **SonarQube:** Análisis completo de calidad y seguridad del código.
- **Bandit:** Análisis estático específico para código Python.

SAST detecta fallos temprano, antes de compilar o desplegar. Se integra en el pipeline de CI/CD.

4.1.2. DAST — Dynamic Application Security Testing

El análisis dinámico prueba la aplicación *mientras está ejecutándose*. Simula ataques reales contra la aplicación en funcionamiento.

**Qué busca:**

- XSS reflejado y almacenado.
- SQL/NoSQL Injection.
- Headers de seguridad inseguros.
- Configuraciones débiles.
- CSRF.
- Problemas de sesión.

Herramientas comunes para DAST:

- **OWASP ZAP:** Scanner de seguridad open source para aplicaciones web.
- **Burp Suite:** Suite de testing de seguridad con proxy interceptador.

4.1.3. Diferencia clave entre SAST y DAST

Aspecto	SAST	DAST
Análisis	Código fuente	Comportamiento en ejecución
Ejecución	No necesita la app	Necesita la app corriendo
Cobertura	Todo el código	Superficie de ataque expuesta
Velocidad	Rápido	Más lento
Fase ideal	Desarrollo	Testing/Pre-producción

Cuadro 8: Comparación SAST vs DAST

Ambos son complementarios. Un análisis completo de seguridad debe incluir ambos enfoques.

4.1.4. Code Review — Revisión de Código

La revisión humana encuentra problemas que ninguna herramienta detecta:

- Errores de lógica de negocio.
- Decisiones de diseño inseguras.
- Patrones que el escáner no reconoce.
- Problemas de arquitectura de seguridad.

Ejemplo de error de lógica que no detecta un escáner:

Listing 6: Lógica potencialmente incorrecta

```

if (user.role == "admin") {
  // Permitir acción administrativa
}
  
```

}

La lógica podría estar mal aunque el código sea sintácticamente correcto. Debería usarse comparación estricta (===) y verificación de la existencia del rol.

Buenas prácticas de code review:

- Revisiones de código obligatorias antes de cada merge.
- Checklist de seguridad en cada Pull Request.
- Mínimo una aprobación antes de merge.
- Revisión especializada para cambios en autenticación y autorización.

4.1.5. Vulnerability Assessment

Escaneo de bibliotecas, dependencias, servidores y contenedores:

- **Escaneo de dependencias:** Herramientas como `npm audit`, `pip-audit` o `safety` identifican dependencias con vulnerabilidades conocidas (CVEs).
- **Escaneo de contenedores:** Herramientas como `trivy` o `docker scan` analizan vulnerabilidades en imágenes Docker.
- **Escaneo de configuraciones:** Verificación de que servidores y servicios estén configurados de forma segura.

4.1.6. Penetration Testing

Un especialista intenta comprometer el sistema de forma controlada. Simula un ataque real y valida si los controles funcionan.

4.2. Acceptance Testing de Seguridad

Es la parte más importante del control 8.29. Antes de pasar a producción se definen criterios claros de aceptación:

Criterio	Objetivo
Vulnerabilidades críticas	0
Vulnerabilidades altas	0
Cobertura SAST	≥ 80 %
Dependencias sin CVEs críticos	100 %
Pruebas ejecutadas	Todas aprobadas
Configuración endurecida	Sí

Cuadro 9: Criterios de aceptación de seguridad

Si el software no cumple con estos criterios, **no se despliega a producción**. Este es el principio fundamental del control 8.29.

4.3. Pipeline CI/CD Seguro

La materialización de los controles 8.25, 8.28 y 8.29 se concreta en un pipeline de integración continua seguro:

Listing 7: Pipeline CI/CD seguro (modelo genérico)

```
Git Push
-> Build (install dependencias)
-> Unit Tests
-> SAST (linter + análisis estático)
-> Dependency Scan (escaneo CVEs)
-> DAST (análisis dinámico)
-> Security Gate (criterios de aceptación)
-> Deploy Staging
-> Acceptance Security Test
-> Production
```

Este pipeline es la materialización práctica de:

- **8.25** → proceso seguro de desarrollo.
- **8.28** → código seguro.
- **8.29** → validación de seguridad.

4.4. Qué busca un auditor en 8.29

Un auditor busca evidencias de:

- Herramientas SAST configuradas y ejecutándose.
- Herramientas DAST configuradas y ejecutándose.
- Registros de análisis de código estático.
- Registros de escaneo de dependencias.
- Criterios de aceptación documentados.
- Evidencia de que el software no se despliega si no cumple criterios.
- Pipeline de CI/CD con Security Gate configurado.

5. Métricas de Calidad

La guía de trabajo práctico exige proponer métricas cuantitativas para la norma ISO/IEC asignada, con sus ecuaciones [ISO/IEC JTC 1/SC 7 \(2026\)](#). Las métricas definidas evalúan los controles 8.25, 8.28 y 8.29 de ISO/IEC 27001:2022.

5.1. Métrica 1: Densidad de Vulnerabilidades

Mide la cantidad de vulnerabilidades encontradas por unidad de código. Es un indicador directo de la calidad de seguridad de la implementación.

$$D_v = \frac{V_{encontradas}}{KLOC} \quad (1)$$

Donde:

- D_v = Densidad de vulnerabilidades (vuln/KLOC).
- $V_{encontradas}$ = Número total de vulnerabilidades encontradas por SAST + DAST.
- $KLOC$ = Miles de líneas de código fuente.

Interpretación:

Rango D_v	Nivel de riesgo
$D_v < 1$	Bajo
$1 \leq D_v < 5$	Medio
$5 \leq D_v < 10$	Alto
$D_v \geq 10$	Crítico

Cuadro 10: Umbrales de densidad de vulnerabilidades

Ejemplo de cálculo: Si un proyecto tiene 10 KLOC y se encuentran 4 vulnerabilidades:

$$D_v = \frac{4}{10} = 0,4 \text{ vuln/KLOC} \Rightarrow \text{Nivel: Bajo} \quad (2)$$

5.2. Métrica 2: Cobertura de Análisis SAST

Mide el porcentaje de archivos del proyecto que fueron analizados por herramientas SAST.

$$C_{SAST} = \frac{A_{escaneados}}{A_{total}} \times 100 \quad (3)$$

Donde:

- C_{SAST} = Cobertura de análisis SAST (%).
- $A_{escaneados}$ = Archivos de código fuente analizados.
- A_{total} = Total de archivos de código fuente.

Objetivo mínimo: $C_{SAST} \geq 80\%$

Ejemplo de cálculo: Si hay 60 archivos de código fuente y se escanean 52:

$$C_{SAST} = \frac{52}{60} \times 100 = 86,7\% \Rightarrow \text{Cumple el objetivo} \quad (4)$$

5.3. Métrica 3: Tiempo Medio de Remediación

Mide el tiempo promedio que tarda el equipo en corregir una vulnerabilidad encontrada.

$$T_{rem} = \frac{\sum_{i=1}^n (t_{corrección_i} - t_{detección_i})}{n} \quad (5)$$

Donde:

- T_{rem} = Tiempo medio de remediación (días).
- $t_{corrección_i}$ = Fecha de corrección de la vulnerabilidad i .
- $t_{detección_i}$ = Fecha de detección de la vulnerabilidad i .
- n = Total de vulnerabilidades corregidas.

Umbrales:

Tiempo	Categoría
$T_{rem} \leq 1$ día	Crítica (corrección inmediata)
$1 < T_{rem} \leq 7$ días	Alta prioridad
$7 < T_{rem} \leq 30$ días	Prioridad media
$T_{rem} > 30$ días	Requiere gestión

Cuadro 11: Umbrales de tiempo de remediación

5.4. Métrica 4: Tasa de Aceptación de Seguridad

Mide el porcentaje de pruebas de seguridad que cumplen con los criterios de aceptación.

$$R_{acep} = \frac{P_{aprobadas}}{P_{total}} \times 100 \quad (6)$$

Donde:

- R_{acep} = Tasa de aceptación de seguridad (%).
- $P_{aprobadas}$ = Pruebas de seguridad que cumplieron criterios.
- P_{total} = Total de pruebas de seguridad ejecutadas.

Objetivo mínimo: $R_{acep} = 100\%$ (todas las pruebas críticas deben aprobar antes de producción).

5.5. Métrica 5: Nivel de Madurez del SSDLC

Mide el nivel de madurez del proceso de seguridad en el ciclo de vida del desarrollo, basado en la escala de ISO/IEC 33000 [ISO/IEC JTC 1/SC 7 \(2026\)](#):

Ecuación de evaluación de madurez:

$$M_{SSDLC} = \frac{\sum_{i=1}^5 P_i \times w_i}{\sum_{i=1}^5 w_i} \quad (7)$$

Donde:

- P_i = Puntaje obtenido en la práctica i (0 o 1).
- w_i = Peso de la práctica i (según prioridad).

Nivel	Nombre	Criterio general
0	Incompleto	No hay proceso de seguridad definido.
1	Realizado	Se realizan pruebas de seguridad de forma ad-hoc.
2	Gestionado	Hay un proceso documentado y se ejecuta de forma controlada.
3	Establecido	El proceso está definido, documentado y se aplica orgánicamente.
4	Predecible	Las métricas de seguridad se miden y controlan cuantitativamente.
5	Optimizado	El proceso se mejora continuamente según métricas y retroalimentación.

Cuadro 12: Escala de madurez SSDLC según ISO/IEC 33000

5.6. Resumen de Métricas

La siguiente tabla resume las cinco métricas propuestas con sus fórmulas y objetivos.

Métrica	Fórmula	Objetivo
Densidad de vulnerabilidades	$D_v = V/KLOC$	$D_v < 1$
Cobertura SAST	$C = A_{esc}/A_{tot} \times 100$	$C \geq 80\%$
Tiempo de remediación	$T_{rem} = \Sigma \Delta t/n$	$T_{rem} \leq 7$ días
Tasa de aceptación	$R = P_{apr}/P_{tot} \times 100$	$R = 100\%$
Madurez SSDLC	$M = \Sigma P_i w_i / \Sigma w_i$	Nivel ≥ 2

Cuadro 13: Resumen de métricas de calidad para los controles 8.25, 8.28 y 8.29

6. Aplicación Práctica y Conclusiones

Esta sección aplica los controles 8.25, 8.28 y 8.29 de ISO/IEC 27001:2022 al proyecto **Workcodile-dev**, una plataforma web full-stack para estudiantes de la Universidad Nacional de Moquegua.

6.1. Contexto del proyecto

Workcodile-dev es una plataforma web para que estudiantes creen, compartan y gestionen cursos y publicaciones. Usa un stack web estándar:

6.2. Aplicación de SSDLC (8.25)

6.2.1. Activos de información

6.2.2. Clasificación de la información

- **Pública:** Cursos disponibles, perfil público de usuarios.

Capa	Tecnología
Frontend	React 18, TypeScript, Vite, Tailwind CSS
Backend	Node.js, Express.js, JWT, bcrypt
Base de datos	MongoDB (NoSQL)
Almacenamiento	MinIO (compatible S3)
Infraestructura	Docker, Docker Compose

Cuadro 14: Stack tecnológico de Workcodile-dev

Activo	Tipo	Ubicación
Base de datos MongoDB	Datos	Backend (contenedor)
Credenciales de usuario	Sensibles	MongoDB (bcrypt hash)
Sesiones JWT	Tokens	Memoria del servidor
Archivos subidos	Datos	MinIO (contenedor)
Variables .env	Secretos	Servidor host
Código fuente	Propiedad	Repositorio Git

Cuadro 15: Activos de información de Workcodile-dev

- **Interna:** Código fuente, configuraciones de Docker, estructura de la base de datos.
- **Confidencial:** Correos electrónicos de usuarios, contraseñas hasheadas, tokens JWT.
- **Restringida:** Variables de entorno con contraseñas de servicios (Gmail, MongoDB, MinIO).

6.2.3. Requisitos de seguridad (CIA + T)

- **Confidencialidad:** Solo usuarios autenticados acceden a sus datos.
- **Integridad:** Los datos de publicaciones no deben ser modificados sin autorización.
- **Disponibilidad:** La aplicación debe estar operativa para los usuarios.
- **Trazabilidad:** Las acciones del sistema deben poder registrarse y auditar.

6.2.4. Threat Modeling con STRIDE

6.2.5. Diagrama de Flujo de Datos (DFD)

La arquitectura de Workcodile-dev presenta los siguientes flujos principales de datos:

1. **Flujo de autenticación:** Usuario → Frontend (React) → Backend (Express) → MongoDB → JWT token.

Amenaza	Ejemplo	Control
Spoofing	Suplantación con JWT falso	Validación firma JWT
Tampering	Modificación datos en tránsito	HTTPS, validación ObjectId
Repudiation	Usuario niega publicación	Logging con timestamp
Info Disclosure	Exposición de .env	.gitignore excluye .env
DoS	Inundación a la API	Rate limiting
Elevation of Priv.	Acceso a rutas admin	Middleware por roles

Cuadro 16: Análisis STRIDE de Workcodile-dev

- Flujo de publicaciones:** Usuario → Frontend → Backend → MinIO (archivo) + MongoDB (metadatos).
- Flujo de correo:** Backend → Nodemailer → Gmail SMTP.

Cada frontera entre componentes representa un límite de confianza que requiere controles adicionales de validación y sanitización.

6.3. Aplicación de Secure Coding (8.28)

6.3.1. Principios aplicados

- Nunca confiar en la entrada:** Validación de ObjectId en cada endpoint que recibe identificadores.

Listing 8: Validación de ObjectId

```
// SEGURO - Con validación de ObjectId
app.get('/api/posts/:id', (req, res) => {
  if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
    return res.status(400).json({ error: 'ID_❌invalido' });
  }
  const post = Post.findById(req.params.id);
  res.json(post);
});
```

- Validar siempre:** Middleware de validación con express-validator en registro.

Listing 9: Middleware de validación

```
const { body, validationResult } = require('express-validator');

app.post('/api/auth/register', [
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 8 })
    .matches(/^(?=.*[A-Z])/),
  body('name').trim().escape()
    .isLength({ max: 100 }),
], (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      errors: errors.array()
    });
  }
});
```


3. **Sanitizar:** React escapa automáticamente el HTML. Se evita el uso de `dangerouslySetInnerHTML`.
4. **Mínimo privilegio:** MongoDB con usuario solo para la BD de la aplicación; MinIO con acceso limitado al bucket; Docker con usuario no root.
5. **Gestión de secretos:** Archivo `.env` excluido de Git, `.env.example` como plantilla versionada.
6. **Manejo seguro de errores:**

Listing 10: Manejo seguro de errores

```
// SEGURO - Mensaje genérico, log interno
app.use((err, req, res, next) => {
  console.error(`${new Date()}] ${err.message}`);
  res.status(500).json({
    error: 'Error interno del servidor'
  });
});
```

7. **Dependencias seguras:** Ejecución periódica de `npm audit` y actualización de paquetes con vulnerabilidades.

6.3.2. OWASP Top 10 — Mapeo

OWASP	¿Aplica?	Controles
Injection	Sí	Validación ObjectId, consultas parametrizadas
Broken Access Control	Sí	Middleware JWT por roles
Cryptographic Failures	Sí	bcrypt, JWT con expiración
Security Misconfiguration	Sí	.env, Docker hardening
Vulnerable Components	Sí	npm audit, actualización deps
Authentication Failures	Sí	JWT expiración, bcrypt cost

Cuadro 17: Mapeo OWASP Top 10 a Workcodile-dev

6.4. Aplicación de Security Testing (8.29)

6.4.1. Herramientas SAST

- **ESLint:** Plugins `eslint-plugin-security` y `eslint-plugin-node` para detección de patrones inseguros.
- **Semgrep:** Análisis estático con reglas para Node.js.
- **SonarQube:** Análisis completo de calidad y seguridad.

6.4.2. Herramientas DAST

- **OWASP ZAP:** Prueba automatizada de la aplicación corriendo.
- **Burp Suite:** Escaneo con proxy interceptador.

6.4.3. Vulnerability Assessment

- `npm audit` — dependencias con vulnerabilidades conocidas.
- `docker scan` — vulnerabilidades en imágenes Docker.
- `trivy` — escaneo completo de contenedores.

6.4.4. Pipeline CI/CD de Workcodile-dev

Listing 11: Pipeline CI/CD de Workcodile-dev

```
Git Push
-> Build (npm install)
-> Unit Tests (Jest / Vitest)
-> SAST (ESLint + Semgrep)
-> Dependency Scan (npm audit)
-> DAST (OWASP ZAP)
-> Security Gate
-> Deploy Staging
-> Acceptance Security Test
-> Production
```

6.5. Métricas aplicadas a Workcodile-dev

Utilizando las fórmulas de la Sección 5 con datos del proyecto:

Métrica	Valor
D_v (Densidad de vulnerabilidades)	$3/5 = 0,6$ (Bajo)
C_{SAST} (Cobertura SAST)	$40/45 \times 100 = 88,9\%$
T_{rem} (Tiempo medio remediación)	5 días (Alta prioridad)
R_{acep} (Tasa de aceptación)	100 % (todas las pruebas críticas pasan)
M_{SSDLC} (Madurez)	Nivel 2 (Gestionado)

Cuadro 18: Métricas de Workcodile-dev

6.6. Conclusiones

1. La aplicación de los controles 8.25, 8.28 y 8.29 de ISO/IEC 27001:2022 al proyecto Workcodile-dev identifica y documenta las prácticas de seguridad existentes y las áreas de mejora.
2. El análisis del SSDLC (8.25) reveló que Workcodile-dev cuenta con controles básicos de seguridad como variables de entorno para secretos (`.env`), autenticación JWT con expiración y hashing de contraseñas con `bcrypt`. Sin embargo, faltan procesos formalizados de threat modeling y documentación de requisitos de seguridad.
3. En el ámbito de Secure Coding (8.28), el proyecto usa tecnologías que protegen por defecto (React escapa HTML automáticamente, Express permite validación de entrada). No obstante, se recomienda implementar plugins de seguridad en ESLint, definir guías de codificación segura y establecer revisiones de código obligatorias con checklist de seguridad.
4. Respecto a las pruebas de seguridad (8.29), actualmente el proyecto solo ejecuta pruebas funcionales con Jest y Vitest. Se requiere incorporar

herramientas SAST (Semgrep, ESLint security), DAST (OWASP ZAP) y escaneo de dependencias (`npm audit`) como parte del pipeline de CI/CD.

5. Las métricas propuestas (densidad de vulnerabilidades, cobertura SAST, tiempo de remediación, tasa de aceptación y nivel de madurez SSDLC) proporcionan un marco cuantitativo para evaluar y mejorar continuamente la seguridad del proceso de desarrollo.
6. Un pipeline CI/CD seguro con Security Gate integra los tres controles: proceso seguro (8.25), código seguro (8.28) y validación de seguridad (8.29).

6.7. Recomendaciones

1. **Corto plazo:** Configurar ESLint con plugins de seguridad, ejecutar `npm audit` y corregir vulnerabilidades críticas identificadas.
2. **Mediano plazo:** Integrar Semgrep y OWASP ZAP en el pipeline de CI/CD, establecer revisiones de código con checklist de seguridad.
3. **Largo plazo:** Implementar un programa completo de Secure SDLC con documentación formal, capacitación del equipo y auditorías periódicas de seguridad.

6.8. Lo que busca un auditor en 8.25, 8.28 y 8.29 — Resumen

Un auditor de ISO/IEC 27001 busca evidencias de:

- Política de Secure SDLC documentada.
- Capacitación del equipo en seguridad.
- Principios de codificación segura aprobados.
- Revisiones de código con hallazgos y correcciones.
- Threat modeling realizado.
- Resultados de SAST y DAST.
- Control de cambios y pipeline seguro.
- Registros de mantenimiento de seguridad.

No busca una herramienta específica. Busca un proceso definido, aplicado y demostrable.

Referencias

ISO/IEC JTC 1/SC 7 (2026). Iso/iec 33000 family: Process assessment. Recuperado de <https://committee.iso.org/sites/jtc1sc7/home/projects/flagship-standards/isoiec-33000-family.html>.

National Institute of Standards and Technology (2022). Nist sp 800-218: Secure software development framework (ssdf) version 1.1. Recuperado de <https://csrc.nist.gov/publications/detail/sp/800-218/final>.

OWASP Foundation (2021). Owasp top 10: 2021. Recuperado de <https://owasp.org/Top10/>.

Wikipedia (2026). Iso/iec 27001: Information security management. Recuperado de https://en.wikipedia.org/wiki/ISO/IEC_27001.